

Queuing Theory

Kyle Bistrain, Brendan Callender

Introduction

Kendall's Notation

- Kendall's notation is a standard system for describing different types of queuing nodes

A general form for the notation is:

$$A/S/c/K/N/D$$

Where:

A describes the distribution for the time between arrivals to the queue

S describes the distribution for the service times in the queue

c is the number of service nodes in the queue

K is the capacity of the queue

N is the size of the population to be served

D is the serving discipline in the queue

- So far in this class we have seen $M/M/k/\infty/\infty/FIFO$
 - Where M/M represents the arrival and service times being memoryless
 - k being the number of servers. We mostly worked with $k = 1$
 - ∞ represents the capacity of the queue and an infinite population for arrivals
 - FIFO representing how the queue serves arrivals on a First-In-First-Out basis
 - This can be simplified to $M/M/k$ which we saw in class

- The most common distribution for arrival times and service times is M for markov or memoryless
 - These are the easiest to work with since we take advantage of the interarrival times being exponentially distributed with the **memoryless property**
 - Other options for distributions include G which stands for general distribution or D which indicates a deterministic time
 - Arrival/Service times having a general distribution makes computations much more difficult.
- Some other serving disciplines other than FIFO include FILO which corresponds to first-in-last-out
 - FILO corresponds to a stack which is a common data structure in computer science

Exercise 1.1. How would we describe the setup for the Costco Application in terms of Kendall's Notation. Remember, the setup for this problem had 6 distinct lines all leading to two different pumps for a total of 6 lines and 12 pumps. Hint: Kendall's Notation describes a node. What is a single node in the context of the problem?

Solution: 6 Distinct Queuing Nodes each with $M/M/2/\infty/\infty/FIFO$ or $M/M/2$

Exercise 1.2. Orders arrive online to a take-out pizza place at a rate of 10 orders per hour. The pizza place has 5 ovens for baking pizzas. Assume all pizzas take 10 minutes to finish baking. What is the setup for this queue in terms of Kendall's Notation?

Solution: $M/D/5/\infty/\infty/FIFO$ or $M/D/5$

M/M/1 Queue with Unreliable Server

- So far in class, we have assumed that a server for a queue will always work perfectly.
- This section explores M/M/1 queues with **unreliable servers**
- **Unreliable servers** have system fails in which the server is unable to service new arrivals
 - Breaks in service are exponentially distributed with some rate parameter γ

- The **repair time** after a server failure, is also exponentially distributed with some rate parameter α

Exercise 3.1. A single server for a website receives requests at a rate of 10 requests per second. The server is able to handle requests at a rate of 20 requests per second. The server goes offline at a rate of once per every 5 minutes and takes on average a minute to be repaired and working again.

- a) Write the arrival, service, failure, and repair rates in terms of minutes.

Solution:

Arrival: $10 \times 60 = 600/\text{min}$

Service: $20 \times 60 = 1,200/\text{min}$

Failure: $1 / 5 = 0.2/\text{min}$

Repair: $1 = 1/\text{min}$

Let S_t be the working status of the server at time t where

$$S_t = \begin{cases} 1, & \text{if the server is working at time } t \\ 0, & \text{if the server is not working at time } t \end{cases}$$

- b) What proportion of time do you expect the server to be online? Hint: How long do you expect the server to be working before it fails vs How long do you expect the server to be down before repair?

Solution:

Average time to failure: 5 minutes (This is when the server is working)

Average time to repair: 1 minute (This is when the server is not working)

Proportion of time working = $5 / (5 + 1) = 83.3\%$

- c) Over the lifetime of the server what is the **effective service rate**? More specifically, what is the service rate of the server after taking into account periods in which the server is down?

Solution:

Service Rate $\times$ Proportion of time server is working

$$0.833 \times 1,200 = 1000 \text{ requests/min}$$

- d) Let N_t be the number of server requests in the system at time t . Do you expect N_t to have a stationary distribution? Why or why not?

Solution:

Yes, because the effective service rate is greater than the arrival rate. Therefore the server is able to manage the incoming requests without getting overwhelmed despite having periods where it isn't working.

- For an M/M/1 queue with an unreliable server with fail rate γ and repair rate α , the **proportion of time the server is operational** is given by:

$$\frac{\alpha}{\alpha + \gamma}$$

- The **effective service rate** is given by:

$$\mu \left(\frac{\alpha}{\alpha + \gamma} \right)$$

Where μ is the service rate of the server when working.

- Notice that when $\gamma = 0$ the effective service rate is equal to the service rate of a normal M/M/1 queue.
- An M/M/1 queue will have a stationary distribution when:

$$\mu \left(\frac{\alpha}{\alpha + \gamma} \right) > \lambda$$

- Additionally, the proportion of time in which an M/M/1 queue is busy (working with requests to handle) is given by:

$$p_{busy} = \frac{\lambda}{\mu}$$

- And the proportion of time in which the system is idle (working with no requests to handle) is given by:

$$p_{idle} = \frac{\alpha}{\alpha + \gamma} - \frac{\lambda}{\mu}$$

Where $\frac{\alpha}{\alpha + \gamma}$ is the proportion of time where the server is working and $\frac{\lambda}{\mu}$ is the proportion of time the server is idle

NOTE: These last two formulas only apply when N_t has a stationary distribution

Exercise 3.2. Customers arrive at McDonalds to order ice cream at a rate of 30 orders every hour. The ice cream machine is able to service orders at a rate of 60 orders every hour. The ice cream machine breaks with a rate of 12 times per day and takes on average 1 hour to repair.

- a) Show that the server has a stationary distribution.

Solution:

Service Rate = 60

Effective Service Rate = $60(1 / (1 + 0.5)) = 40$

$40 > 30$ meaning the effective service rate is greater than the arrival rate so the queue has a stationary distribution

- b) What proportion of time is at least one person waiting to receive ice cream?

Solution:

$30 / 60 = 50\%$ of the time

c) What proportion of time is the ice cream machine sitting idle with no orders?

Solution:

$$(1 / (1 + 0.5)) - 0.5 = 16.67\% \text{ of the time}$$

Unreliable Server Simulation

NOTE FOR DR. ROSS: These simulations take very long to converge when the rates for the arrival times and service times are much larger than the rates for failure and repair. So for the purposes of the simulations we try to maintain the ratio between the arrival and service rates but make them much smaller to get more accurate results with less simulation

- In this section we will be using simulation to compare a regular M/M/1 queue and an M/M/1 queue with an unreliable server
- Using properties of exponential distributions, we can easily write code to simulate an M/M/1 queue
 - `get_next_event` uses properties of exponential races to simulate the next event to happen in the queue

```
get_next_event <- function(in_system, lambda, mu) {  
  if (in_system > 0) {  
    t <- rexp(1, lambda + mu)  
    names(t) <- sample(c("arrival", "service"),  
                      prob = c(lambda/(lambda+mu), mu/(lambda+mu)),  
                      size = 1, replace = TRUE)  
  } else {  
    t <- rexp(1, lambda)  
    names(t) <- "arrival"  
  }  
  t  
}
```

```

sim_m_m_1 <- function(n_iter, lambda, mu) {
  in_system <- 0
  current_time <- 0

  N <- numeric(n_iter); N[1] <- in_system
  t <- numeric(n_iter); t[1] <- current_time

  for (i in 2:n_iter) {
    next_event <- get_next_event(in_system, lambda, mu)
    current_time <- current_time + unname(next_event)
    in_system <- if_else(names(next_event) == "arrival", in_system + 1, in_system - 1)

    N[i] <- in_system
    t[i] <- current_time
  }
  data.frame(t, N)
}

```

```

get_avg_N <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t, 1),
           t_int = t_end - t,
           weighted_N = N*t_int) %>%
    filter(!is.na(t_int)) %>%
    summarize(mean_N = sum(weighted_N)/total_time)
}

```

```

get_prop_busy <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t, 1),
           t_int = t_end - t) %>%
    filter(N > 0 & !is.na(t_int)) %>%
    summarize(prop_busy = sum(t_int)/total_time)
}

```

```

get_prop_idle <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t, 1),
           t_int = t_end - t) %>%

```

```

    filter(N == 0 & !is.na(t_int)) %>%
    summarize(prop_idle = sum(t_int)/total_time)
}

```

- Simulating an M/M/1 with an unreliable server is similar but adds a little more complexity
- The code used is shown below
- Notice `get_next_event_us` is much more complicated than `get_next_event` due to the queue having different states depending on whether it is working or offline

```

get_next_event_us <- function(in_system, is_working, lambda, mu, gamma, alpha) {
  if (is_working) {
    if (in_system > 0) {
      t <- rexp(1, lambda + mu + gamma)
      names(t) <- sample(c("arrival", "service", "failure"),
                        prob = c(lambda/(lambda+mu+gamma), mu/(lambda+mu+gamma), gamma/(lambda+mu+gamma)),
                        size = 1, replace = TRUE)
    } else {
      t <- rexp(1, lambda + gamma)
      names(t) <- sample(c("arrival", "failure"),
                        prob = c(lambda/(lambda+gamma), gamma/(lambda+gamma)),
                        size = 1, replace = TRUE)
    }
  } else {
    t <- rexp(1, lambda + alpha)
    names(t) <- sample(c("arrival", "repaired"),
                      prob = c(lambda/(lambda+alpha), alpha/(lambda+alpha)),
                      size = 1, replace = TRUE)
  }
  t
}

sim_m_m_1_us <- function(n_iter, lambda, mu, gamma, alpha) {
  in_system <- 0
  is_working <- 1
  current_time <- 0

  N <- numeric(n_iter); N[1] <- in_system

```

```

S <- numeric(n_iter); S[1] <- is_working
t <- numeric(n_iter); t[1] <- current_time

for (i in 2:n_iter) {
  next_event <- get_next_event_us(in_system, is_working, lambda, mu, gamma, alpha)
  current_time <- current_time + unname(next_event)
  in_system <- case_when(names(next_event) == "arrival" ~ in_system + 1,
                        names(next_event) == "service" ~ in_system - 1,
                        TRUE ~ in_system)
  is_working <- if_else(names(next_event) == "failure", 0, 1)
  N[i] <- in_system
  S[i] <- is_working
  t[i] <- current_time
}
data.frame(t, N, S)
}

```

```

get_prop_working <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t,1),
           t_int = t_end-t,
           weight_S = S*t_int) %>%
    filter(!is.na(weight_S)) %>%
    summarize(prop_working = sum(weight_S)/total_time)
}

```

Exercise 3.3. A single server for a website receives requests at a rate of 10 requests per second. The server is able to handle requests at a rate of 20 requests per second. The server goes offline at a rate of once per every 5 minutes and takes on average a minute to be repaired and working again. Use simulation to confirm the answers from parts b of **Exercise 3.1**

Solution: We see the simulated proportion of 0.83 matches what we calculated in part b

```

sim <- sim_m_m_1_us(50000, 1/10, 1/5, 1/5, 1)
sim <- sim[1000:nrow(sim),] # burn period

get_prop_working(sim)

```

```

prop_working
1      0.8297057

```

```

get_prop_busy_us <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t, 1),
           t_int = t_end - t) %>%
    filter(N > 0 & !is.na(t_int) & S == 1) %>%
    summarize(prop_busy = sum(t_int)/total_time)
}

get_prop_idle_us <- function(sim) {
  total_time <- sim$t[nrow(sim)]
  sim %>%
    mutate(t_end = lead(t, 1),
           t_int = t_end - t) %>%
    filter(N == 0 & !is.na(t_int) & S == 1) %>%
    summarize(prop_idle = sum(t_int)/total_time)
}

```

Exercise 3.4. Customers arrive at McDonalds to order ice cream at a rate of 30 orders every hour. The ice cream machine is able to service orders at a rate of 60 orders every hour. The ice cream machine breaks with a rate of 12 times per day and takes on average 1 hour to repair. Use simulation to verify the answers from parts b and c in **Exercise 3.2**.

Solution: Both of the simulated values match their respective theoretical values of 0.5 and 0.167 which were calculated in Exercise 3.2.

```

sim <- sim_m_m_1_us(50000, 1/10, 1/5, 1/2, 1)
sim <- sim[5000:nrow(sim),] # burn period

```

```
get_prop_busy_us(sim)
```

```

prop_busy
1 0.4484374

```

```
get_prop_idle_us(sim)
```

```

prop_idle
1 0.1707047

```

Exercise 3.5 Pick some configuration for the arrival rates and service rates and compare how i) the average number of tasks in the system, ii) proportion of time the system is busy, and iii) the proportion of time the system is idle between a regular M/M/1 queue and the same queue with but with an unreliable server.

```
sim_regular <- sim_m_m_1(10000, 1/10, 1/2)
sim_us <- sim_m_m_1_us(10000, 1/10, 1/2, 1/20, 1/30)
```

i)

The average number of requests in the unreliable server is greater than the number of requests in the regular queue.

```
c(unlist(get_avg_N(sim_regular)), unlist(get_avg_N(sim_us)))
```

```
      mean_N      mean_N
0.2591892 0.3722265
```

ii)

The proportion of time where the system is busy and has tasks to deal with is the same for both the regular and unreliable server queues. This makes sense because the periods in which the server is offline are not counted as the system being busy.

```
c(unlist(get_prop_busy(sim_regular)), unlist(get_prop_busy_us(sim_us)))
```

```
prop_busy prop_busy
0.2039614 0.1915609
```

iii)

The proportion of time in which the system has no tasks is greater for the regular queue than the unreliable server queue. This makes sense because the us queue has to catch up on tasks after going failing and waiting to be repaired.

```
c(unlist(get_prop_idle(sim_regular)), unlist(get_prop_idle_us(sim_us)))
```

```
prop_idle prop_idle
0.7960386 0.5327560
```

Exercise 3.6 See how the different estimates from the last question change as you change the values for the failure and repair rates.

Base Performance

```
c(unlist(get_avg_N(sim_us)), unlist(get_prop_busy_us(sim_us)), unlist(get_prop_idle_us(sim_us)))
```

```
      mean_N prop_busy prop_idle  
0.3722265 0.1915609 0.5327560
```

Increasing rate of failure

```
inc_fail_rate <- sim_m_m_1_us(10000, 1/10, 1/2, 1/15, 1/30)  
c(unlist(get_avg_N(inc_fail_rate)), unlist(get_prop_busy_us(inc_fail_rate)), unlist(get_prop_idle_us(inc_fail_rate)))
```

```
      mean_N prop_busy prop_idle  
0.3939427 0.1963940 0.4734915
```

Decreasing rate of failure

```
dec_fail_rate <- sim_m_m_1_us(10000, 1/10, 1/2, 1/25, 1/30)  
c(unlist(get_avg_N(dec_fail_rate)), unlist(get_prop_busy_us(dec_fail_rate)), unlist(get_prop_idle_us(dec_fail_rate)))
```

```
      mean_N prop_busy prop_idle  
0.3658054 0.2044332 0.5676779
```

Increasing rate of repair

```
inc_rep_rate <- sim_m_m_1_us(10000, 1/10, 1/2, 1/20, 1/20)  
c(unlist(get_avg_N(inc_rep_rate)), unlist(get_prop_busy_us(inc_rep_rate)), unlist(get_prop_idle_us(inc_rep_rate)))
```

```
      mean_N prop_busy prop_idle  
0.3582735 0.1998387 0.5557332
```

Decreasing rate of repair

```
dec_rep_rate <- sim_m_m_1_us(10000, 1/10, 1/2, 1/20, 1/40)  
c(unlist(get_avg_N(dec_rep_rate)), unlist(get_prop_busy_us(dec_rep_rate)), unlist(get_prop_idle_us(dec_rep_rate)))
```

```
      mean_N prop_busy prop_idle  
0.3753671 0.1956770 0.5232466
```

M/M/1 Priority Queuing Theory

Suppose we have a M/M/1 System where tasks are executed for customers in a single server or channel. Some tasks have a higher priority than others.

Assumptions:

- We assume that all processing times are random from an exponential distribution with a mean $1/\mu$, μ is the service rate
- $\rho_1 + \rho_2 + \dots + \rho_r < 1$, where r is the number of task types (hence, the total task workload must be lower than server capacity)
- $\rho_i = \frac{\lambda_i}{\mu}$, where λ_i is the arrival rate for the type of task i

Definitions:

- W_i : The waiting time for specific task i
- S_i : (Sojourn Time) Total time a specific task i stays in the System (Waiting and Service time)
- B_i : The service time for a specific task i
- L_i : Number of tasks i in the the server.

Preemptive-Resume Priority Queue

In the preemptive-resume case, higher-priority tasks have “absolute” priority over lower-priority tasks.

- If a lower-priority task is being processed and a higher-priority task arrives in the queue, the lower priority task the server stops the lower-priority task immediately. It then processes all the higher-priority tasks waiting in the queue first before processing the lower-priority tasks.

Mean Waiting Time $E(W_i)$:

$$E(W_i) = \frac{\sum_{j=1}^i \rho_j / \mu}{(1 - (\rho_1 + \dots + \rho_i))(1 - (\rho_1 + \dots + \rho_{i-1}))}$$

$$\forall i = 1, \dots, r$$

Mean Sojourn Time $E(S_i)$:

$$E(S_i) = \frac{\sum_{j=1}^i \rho_j / \mu}{(1 - (\rho_1 + \dots + \rho_i))(1 - (\rho_1 + \dots + \rho_{i-1}))} + \frac{1/\mu}{1 - (\rho_1 + \dots + \rho_{i-1})}$$

$$\forall i = 1, \dots, r$$

Mean Service Time $E(B_i)$:

$$E(B_i) = \frac{1}{\mu}$$

Mean Number of Tasks i in the system $E(L_i)$:

$$E(L_i) = \lambda_i E(S_i) = \lambda_i \left[\frac{\sum_{j=1}^i \lambda_j / \mu^2}{(1 - (\rho_1 + \dots + \rho_i))(1 - (\rho_1 + \dots + \rho_{i-1}))} + \frac{1/\mu}{1 - (\rho_1 + \dots + \rho_{i-1})} \right]$$

$$\forall i = 1, \dots, r$$

(Little's Law) Mean Number of Tasks i in the queue $E(L_i^q)$:

$$E(L_i^q) = \lambda_i E(W_i)$$

Non-Preemptive Priority Queue

Using a non-preemptive priority is more practical because interrupting tasks in progress creates additional inefficiencies. In the non-preemptive priority situation, once a task is processing, it completes service before any other order (even a higher priority one) can begin service.

Mean Waiting Time $E(W_i)$:

$$E(W_i) = \frac{\sum_{j=1}^r \rho_j / \mu}{(1 - (\rho_1 + \dots + \rho_{i-1}))(1 - (\rho_1 + \dots + \rho_i))}$$

$$\forall i = 1, \dots, r$$

Mean Sojourn Time $E(S_i)$:

$$E(S_i) = \frac{\sum_{j=1}^r \rho_j / \mu}{(1 - (\rho_1 + \dots + \rho_{i-1}))(1 - (\rho_1 + \dots + \rho_i))} + \frac{1}{\mu}$$

$$\forall i = 1, \dots, r$$

Mean Service Time $E(B_i)$:

$$E(B_i) = \frac{1}{\mu}$$

Expected Number of Tasks i in the System $E(L_i)$:

$$E(L_i) = \lambda_i E(S_i) = \lambda_i \left[\frac{\sum_{j=1}^r \lambda_j / \mu^2}{(1 - (\rho_1 + \dots + \rho_{i-1}))(1 - (\rho_1 + \dots + \rho_i))} + \frac{1}{\mu} \right]$$

$$\forall i = 1, \dots, r$$

(Little's Law) Mean Number of Tasks i in the queue $E(L_i^q)$:

$$E(L_i^q) = \lambda_i E(W_i)$$

$$\forall i = 1, \dots, r$$

Exercise 4.1. A stock market exchange uses a preemptive-resume priority queue to process orders. Market Maker orders (type 1) arrive at a rate of $\lambda_{MarketMaker} = 0.7$ orders per nanosecond, and Retail Trader orders (type 2) arrive at $\lambda_{RetailTrader} = 0.1$ orders per nanosecond. The server processes orders at a rate of $\mu = 2$ orders per nanosecond. Market Makers have higher priority and will halt a Retail Trader's order if processing and the Market Maker submits a trade order before it is complete.

- a) What are the expected total times until the transaction is completed for a Market Maker and a Retail Trader?
- b) What is the expected number of Retail Traders in the queue?
- c) What is the overall mean time from entering the queue to an executed trade?

Exercise 4.2. A stock market exchange uses a non-preemptive priority queue to process orders. Market Maker orders (type 1) arrive at a rate of $\lambda_{MarketMaker} = 0.78$ orders per nanosecond, and Retail Trader orders (type 2) arrive at $\lambda_{RetailTrader} = 0.2$ orders per nanosecond. The server processes orders at a rate of $\mu = 3$ orders per nanosecond. Market Makers have higher priority and can't halt Retail Traders while the order is being processed.

- a) What are the expected total time from transaction initiation to completion for a Market Maker and Retail Trader?
 - b) What is the expected number of Retail Traders in the system?
 - c) What is the overall mean time from entering the queue to an executed trade?
-

Simulation-based and Analytical Solutions

Simulation

```
set.seed(1334)
library(simmer)
```

Attaching package: 'simmer'

The following objects are masked from 'package:lubridate':

now, rollback

The following object is masked from 'package:dplyr':

select

The following object is masked from 'package:tidyr':

separate

```

lambda1 <- 0.7
lambda2 <- 0.1
mu <- 2
sim_time <- 1000000
rho_1 <- lambda1 / mu

preemptive.sim <- simmer("preemptive")

market_maker <- trajectory("Market Maker") %>%
  seize("server") %>%
  timeout(function() rexp(1, mu)) %>%
  release("server")

retail_trader <- trajectory("Retail Trader") %>%
  seize("server") %>%
  timeout(function() rexp(1, mu)) %>%
  release("server")

preemptive.sim %>%
  add_resource("server", 1, preemptive = TRUE) %>%
  add_generator("MM_", market_maker, function() rexp(1, lambda1), priority = 1) %>%
  add_generator("RT_", retail_trader, function() rexp(1, lambda2), priority = 0) %>%
  run(until = sim_time)

```

```

simmer environment: preemptive | now: 1e+06 | next: 1000000.84745809
{ Monitor: in memory }
{ Resource: server | monitored: TRUE | server status: 0(1) | queue status: 0(Inf) }
{ Source: MM_ | monitored: 1 | n_generated: 699113 }
{ Source: RT_ | monitored: 1 | n_generated: 99785 }

```

```

arrivals <- get_mon_arrivals(preemptive.sim)

mm.arrivals <- arrivals[grep("MM_", arrivals$name), ]

E_S1_sim <- mean(mm.arrivals$end_time - mm.arrivals$start_time)

rt_arrivals <- arrivals[grep("RT_", arrivals$name), ]

E_S2_sim <- mean(rt_arrivals$end_time - rt_arrivals$start_time)

E_Wq2_sim <- E_S2_sim - (1 / mu) / (1 - rho_1)

```

```

L2_sim <- lambda2 * E_S2_sim

Lq2_sim <- lambda2 * E_Wq2_sim

E_S_sim <- (lambda1 * E_S1_sim + lambda2 * E_S2_sim) / (lambda1 + lambda2)

```

```

set.seed(1334)
library(simmer)

lambda1 <- 0.78
lambda2 <- 0.2
mu <- 3
sim_time <- 2000000

nonpreemptive.sim <- simmer("nonpreemptive")

market_maker <- trajectory("Market Maker") %>%
  seize("server") %>%
  timeout(function() rexp(1, mu)) %>%
  release("server")

retail_trader <- trajectory("Retail Trader") %>%
  seize("server") %>%
  timeout(function() rexp(1, mu)) %>%
  release("server")

nonpreemptive.sim %>%
  add_resource("server", 1, preemptive = FALSE) %>%
  add_generator("MM_", market_maker, function() rexp(1, lambda1), priority = 1) %>%
  add_generator("RT_", retail_trader, function() rexp(1, lambda2), priority = 0) %>%
  run(until = sim_time)

```

```

simmer environment: nonpreemptive | now: 2e+06 | next: 2000001.49392315
{ Monitor: in memory }
{ Resource: server | monitored: TRUE | server status: 0(1) | queue status: 0(Inf) }
{ Source: MM_ | monitored: 1 | n_generated: 1558999 }
{ Source: RT_ | monitored: 1 | n_generated: 400207 }

```

```

arrivals <- get_mon_arrivals(nonpreemptive.sim)

mm.arrivals <- arrivals[grep("MM_", arrivals$name), ]

```

```

E_S1_sim2 <- mean(mm.arrivals$end_time - mm.arrivals$start_time)

rt_arrivals <- arrivals[grepl("RT_", arrivals$name), ]

E_S2_sim2 <- mean(rt_arrivals$end_time - rt_arrivals$start_time)

E_Wq2_sim2 <- E_S2_sim2 - (1 / mu)

L2_sim2 <- lambda2 * E_S2_sim2

Lq2_sim2 <- lambda2 * E_Wq2_sim2

E_S_sim2 <- (lambda1 * E_S1_sim2 + lambda2 * E_S2_sim2) / (lambda1 + lambda2)

```

Excercise 4.1.

a)

```
E_S1_sim
```

```
[1] 0.7671843
```

```
E_S2_sim
```

```
[1] 1.26637
```

b)

```
Lq2_sim
```

```
[1] 0.0497139
```

c)

```
E_S_sim
```

```
[1] 0.8295825
```

Excercise 4.2.

a)

```
E_S1_sim2
```

```
[1] 0.4805428
```

```
E_S2_sim2
```

```
[1] 0.5517766
```

b)

```
L2_sim2
```

```
[1] 0.1103553
```

c)

```
E_S_sim2
```

```
[1] 0.4950803
```

Analytical Solution

Excercise 4.1.

a)

$$E(S_1) = \frac{\sum_{j=1}^1 \rho_j / \mu}{(1 - (\rho_1 + \rho_2))(1 - \rho_1)} + \frac{1}{\mu}$$

where

$$\rho_1 = \frac{0.7}{2} = 0.35$$

Therefore:

$$E(S_2) = \frac{\frac{.35}{2}}{(0.65)} + 0.5$$

$$= 0.2692308 + .5 = 0.76923$$

Thus, the $E(S_1)$ for Market Makers is **.769** nanoseconds.

$$E(S_2) = \frac{\sum_{j=1}^2 \rho_j / \mu}{(1 - (\rho_1 + \rho_2))(1 - \rho_1)} + \frac{1/\mu}{1 - \rho_1}$$

where

$$\rho_1 = \frac{0.7}{2} = 0.35, \quad \rho_2 = \frac{0.1}{2} = 0.05$$

Therefore:

$$\begin{aligned} E(S_2) &= \frac{(0.35 + 0.05)/2}{(0.6)(0.65)} + \frac{0.5}{0.65} \\ &= \frac{0.2}{0.39} + 0.769 = 0.513 + 0.769 = 1.28 \end{aligned}$$

Thus, the $E(S_2)$ for Retail Traders is **1.28** nanoseconds.

b)

$$E(L_2^q) = \lambda_2 E(W_2)$$

$$= \lambda_2 (.5128)$$

$$= .1(.5128) = .05128$$

Thus, the $E(L_2^q)$ for Retail Traders is **.05128** nanoseconds.

c)

$$\begin{aligned}
E(S) &= \frac{\lambda_1 E(S_1) + \lambda_2 E(S_2)}{\lambda_1 + \lambda_2} \\
&= \frac{.7(.769) + .1(1.28)}{.7 + .1} \\
&= .833
\end{aligned}$$

Thus, The mean time from trade initiation to completion $E(S)$ is **.833** nanoseconds regardless of the trader type.

Excercise 4.2.

a)

$$E(S_1) = \frac{\sum_{j=1}^1 \rho_j / \mu}{(1 - \rho_1)} + (1/\mu)$$

where

$$\rho_1 = \frac{0.78}{3} = 0.26$$

Therefore:

$$\begin{aligned}
E(S_2) &= \frac{\frac{.26 + .06666}{3}}{.74} + \frac{1}{3} \\
&= .48048
\end{aligned}$$

Thus, the $E(S_1)$ for Market Makers is **.4805** nanoseconds.

$$E(S_2) = \frac{\sum_{j=1}^2 \rho_j / \mu}{(1 - \rho_1)(1 - (\rho_1 + \rho_2))} + \frac{1}{\mu}$$

where

$$\rho_1 = \frac{0.78}{3} = 0.26, \quad \rho_2 = \frac{0.2}{3} = 0.06666$$

Therefore:

$$\begin{aligned}
 E(S_2) &= \frac{\frac{(0.26+0.06666)}{3}}{(0.74)(.67334)} + \frac{1}{3} \\
 &= \frac{0.10888}{.4983} + \frac{1}{3} = .552
 \end{aligned}$$

Thus, the $E(S_2)$ for Retail Traders is **.552** nanoseconds.

b)

$$\begin{aligned}
 E(L_2) &= \lambda_2 E(S_2) \\
 &= .2(.552) \\
 &= .1104
 \end{aligned}$$

Thus, the $E(L_2)$ for Retail Traders is **.1104** nanoseconds.

c)

$$\begin{aligned}
 E(S) &= \frac{\lambda_1 E(S_1) + \lambda_2 E(S_2)}{\lambda_1 + \lambda_2} \\
 &= \frac{.78(.4805) + .2(.552)}{.78 + .2} \\
 &= .49509
 \end{aligned}$$

Thus, The mean time from trade initiation to completion $E(S)$ is **.495** nanoseconds regardless of the trader type.